Mudcard

- **Doesn't k nearest neighbors have a weight parameter?**
  - yes but it doesn't weight the nearest neighbors with respect to the classification label but with respect to how far a nearest neighbor is from the point you are trying to predict
- **For imbalanced, is the model only worth it if the metrics are above 99% for example, since the baseline in real data (assuming its 99% to 1% balance) is going to be 99%?**
  - Correct.
  - If accuracy is your evaluation metric, a model is only worth considering for deployment if its accuracy is above the baseline.
  - Otherwise what's the point? Your accuracy would be higher if you predicted class 0 to all points
- **is it ever worth resampling/rebalancing?**
  - yes, absolutely but only if resampling improves the model's performance on the original class distribution

Non-linear ML algorithms - Support Vector Machines

By the end of this lecture, you will be able to

- Describe how linear SVMs work and what support vectors are
- Apply non-linear SVMs with radial basis functions (gaussian kernel)
- Discuss what secondary considerations might go into selecting/excluding ML models

Non-linear ML algorithms - Support vector machines

By the end of this lecture, you will be able to

- **Describe how linear SVMs work and what support vectors are**
- Apply non-linear SVMs with radial basis functions (gaussian kernel)
- Discuss what secondary considerations might go into selecting/excluding ML models

Linear Support Vector Machines

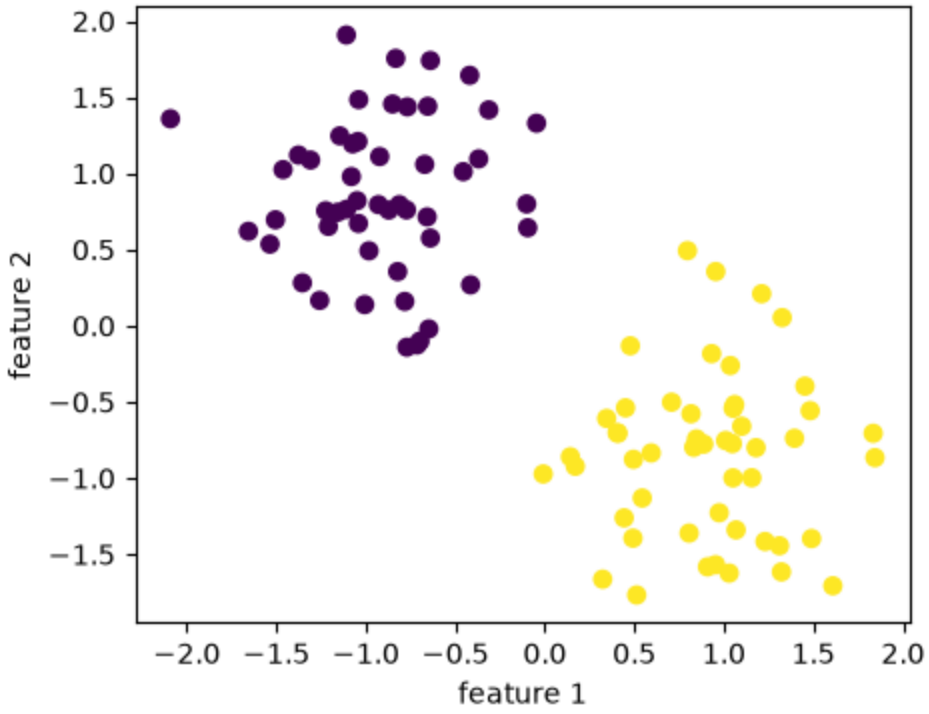
- let's consider a simple binary classification problem with two features
- the two classes are linearly separable meaning that you can find a linear decision boundary which gives 0 error on the training set
- here is an example:

In [1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib
from sklearn.datasets import make_blobs
from sklearn.preprocessing import StandardScaler


X_train, y_train = make_blobs(n_samples=100, n_features = 2, centers = 2, cl

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)

plt.figure(figsize=(5,4))
plt.scatter(X_train[:, 0], X_train[:, 1], c=y_train)
plt.xlabel('feature 1')
plt.ylabel('feature 2')
plt.show()
```



Based on the logistic regression lecture, we can draw linear decision boudnaries for this dataset.

In [2]:
```
# create the w vector:
w = np.zeros([2])

phi = -0.9 # [radians] any value between -0.1 and -0.9 gives 0 loss on the t
length = 1

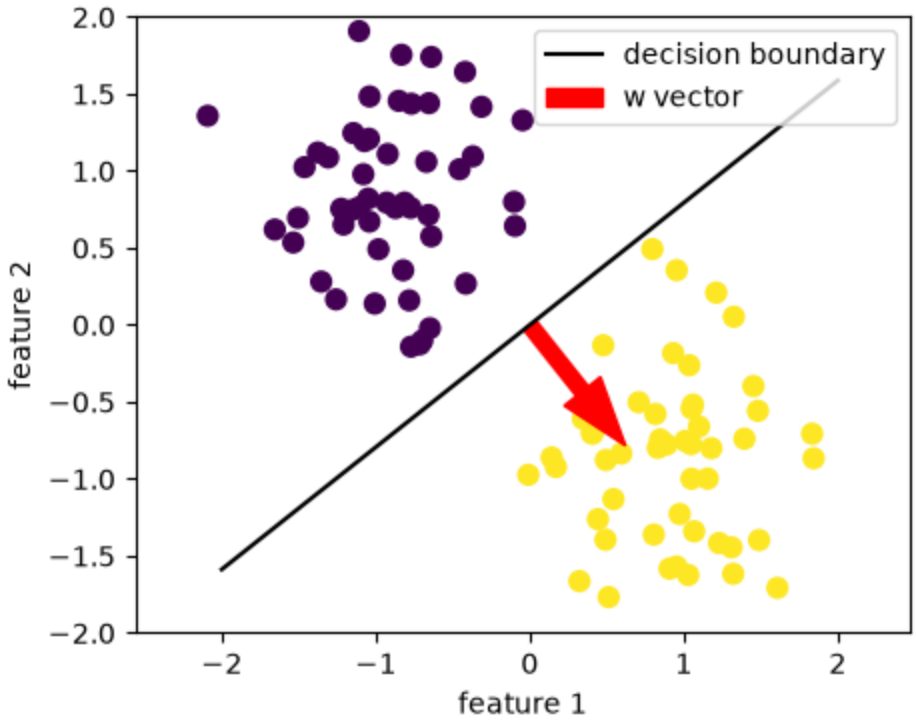
w[0] = np.cos(phi)*length
w[1] = np.sin(phi)*length

# calculate the decision boundary - it's a line perpendicular to the w vecto
a = -w[0] / w[1]
xx = np.linspace(-2, 2)
yy = a * xx

plt.figure(figsize=(5,4))
plt.axis('equal')
plt.ylim((-2,2))
# visualize the decision boundary
plt.plot(xx, yy, "k-",label='decision boundary')

# add the datapoints
plt.scatter(X_train[:,0],X_train[:,1],c=y_train,s=50)
plt.xlabel('feature 1')
plt.ylabel('feature 2')

# add the w vector
plt.arrow(0,0,w[0],w[1],width=0.1,length_includes_head=True,color='r',label=
plt.legend()
plt.show()
```

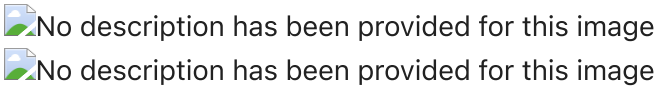


- any phi between -0.1 and -0.9 radians give a 0 loss on the training data!

- which one should we choose?
- not all decision boundaries are created equal!
- margin helps us answer this question

Margin

- the minimum distance between any training point and the decision boundary
- light blue region on the figures



- out of the inifnite number of decision boundaries we could choose, we want one that maximizes the margin (example 2 in this case)!
- this is the key idea of a support vector machine

What are the support vectors?

- **The support vectors are data points that define the margin**
- each data point is a vector in the feature space
- the points that 'support' or define the margin are the support vectors
- the other points don't matter because the decision boundary and the margin are only impacted by the support vectors!
- let's visualize the linear SVM and the support vectors on the toy dataset:

In [3]:
```
from sklearn.svm import SVC

clf = SVC(kernel='linear',C=1e10) # linear SVM with no or very little regula
clf.fit(X_train,y_train)

plt.figure(figsize=(5,4))
plt.axis('equal')

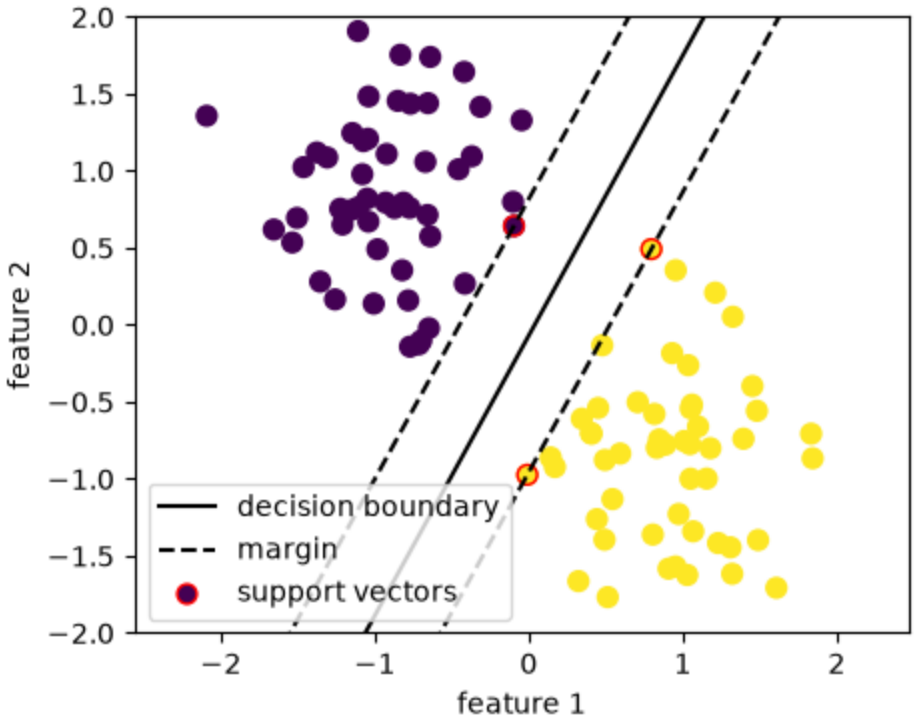
# get the separating hyperplane
w = clf.coef_[0] # this is the w vector
a = -w[0] / w[1]
xx = np.linspace(-2, 2)
yy = a * xx - (clf.intercept_[0]) / w[1]

# plot the parallels to the separating hyperplane that pass through the
# support vectors (margin away from hyperplane in direction
# perpendicular to hyperplane). This is sqrt(1+a^2) away vertically in
# 2-d.
margin = 1 / np.sqrt(np.sum(clf.coef_**2))
yy_down = yy - np.sqrt(1 + a**2) * margin
yy_up = yy + np.sqrt(1 + a**2) * margin

plt.ylim((-2,2))
```

```
plt.plot(xx, yy, "k-",label='decision boundary')
plt.plot(xx, yy_down, "k--",label='margin')
plt.plot(xx, yy_up, "k--")

# add the datapoints
plt.scatter(X_train[:,0],X_train[:,1],c=y_train,s=50)
# clf.support_ is the indices of the support vectors!
plt.scatter(X_train[clf.support_,0],X_train[clf.support_,1],c=y_train[clf.su
plt.xlabel('feature 1')
plt.ylabel('feature 2')
plt.legend(loc=3)
plt.show()
```



Quiz 1

How many support vectors are there in the example below?

No description has been provided for this image

Non-linear ML algorithms - Support vector machines

By the end of this lecture, you will be able to

- Describe how linear SVMs work and what support vectors are
- **Apply non-linear SVMs with radial basis functions (gaussian kernel)**

- Discuss what secondary considerations might go into selecting/excluding ML models

Non-linear SVMs

- non-linear SVMs are an extension of linear SVMs
- The idea is to map or embed the data points into a higher (sometimes infinite) dimensional feature space where the problem is linearly separable
  - think polynomial regression in lecture 8!
- We train a linear SVM in this higher dimensional space
- It can be very computationally expensive to perform feature transformations and to train a linear SVM in a very high dimensional space.
  - if the feature space is infinite dimensional, this is in fact impossible to do computationally
- A mathematical trick called the **kernel trick** is applied to perform these operations in a computationally efficient manner
- the math is pretty complex and beyond this course
- recommend CSCI1420 or DATA2060 to dive deeper into ML algorithms
- If you don't plan to take those courses but want to learn more, check out this post for a high level overview

Gaussian SVMs

- SVM with a Gaussian or radial basis function kernel
- popular choice among possible non-linear SVM kernels
- this kernel basically smoothes datapoints to multi-dimensional gaussians

SVM in regression

In [4]:
```
import numpy as np
from sklearn.svm import SVR
np.random.seed(10)
def true_fun(X):
    return np.cos(1.5 * np.pi * X)

n_samples = 30

X = np.random.rand(n_samples)
y = true_fun(X) + np.random.randn(n_samples) * 0.1

X_new = np.linspace(-0.5, 1.5, 2000)

reg = SVR(gamma = 1, C = 1)
reg.fit(X[:, np.newaxis],y)
y_new = reg.predict(X_new[:, np.newaxis])
```

In [5]:
```
help(SVR)
```

```
Help on class SVR in module sklearn.svm._classes:

class SVR(sklearn.base.RegressorMixin, sklearn.svm._base.BaseLibSVM)
 |  SVR(
 |      *,
 |      kernel='rbf',
 |      degree=3,
 |      gamma='scale',
 |      coef0=0.0,
 |      tol=0.001,
 |      C=1.0,
 |      epsilon=0.1,
 |      shrinking=True,
 |      cache_size=200,
 |      verbose=False,
 |      max_iter=-1
 |  )
 |
 |  Epsilon-Support Vector Regression.
 |
 |  The free parameters in the model are C and epsilon.
 |
 |  The implementation is based on libsvm. The fit time complexity
 |  is more than quadratic with the number of samples which makes it hard
 |  to scale to datasets with more than a couple of 10000 samples. For large
 |  datasets consider using :class:`~sklearn.svm.LinearSVR` or
 |  :class:`~sklearn.linear_model.SGDRegressor` instead, possibly after a
 |  :class:`~sklearn.kernel_approximation.Nystroem` transformer or
 |  other :ref:`kernel_approximation`.
 |
 |  Read more in the :ref:`User Guide <svm_regression>`.
 |
 |  Parameters
 |  ----------
 |  kernel : {'linear', 'poly', 'rbf', 'sigmoid', 'precomputed'} or callabl
e,          default='rbf'
 |       Specifies the kernel type to be used in the algorithm.
 |       If none is given, 'rbf' will be used. If a callable is given it is
 |       used to precompute the kernel matrix.
 |       For an intuitive visualization of different kernel types
 |       see :ref:`sphx_glr_auto_examples_svm_plot_svm_regression.py`
 |
 |  degree : int, default=3
 |      Degree of the polynomial kernel function ('poly').
 |      Must be non-negative. Ignored by all other kernels.
 |
 |  gamma : {'scale', 'auto'} or float, default='scale'
 |      Kernel coefficient for 'rbf', 'poly' and 'sigmoid'.
 |
 |      - if ``gamma='scale'`` (default) is passed then it uses
 |        1 / (n_features * X.var()) as value of gamma,
 |      - if 'auto', uses 1 / n_features
 |      - if float, must be non-negative.
 |
 |      .. versionchanged:: 0.22
 |         The default value of ``gamma`` changed from 'auto' to 'scale'.
```

```
|
|  coef0 : float, default=0.0
|      Independent term in kernel function.
|      It is only significant in 'poly' and 'sigmoid'.
|
|  tol : float, default=1e-3
|      Tolerance for stopping criterion.
|
|  C : float, default=1.0
|      Regularization parameter. The strength of the regularization is
|      inversely proportional to C. Must be strictly positive.
|      The penalty is a squared l2. For an intuitive visualization of the
|      effects of scaling the regularization parameter C, see
|      :ref:`sphx_glr_auto_examples_svm_plot_svm_scale_c.py`.
|
|  epsilon : float, default=0.1
|       Epsilon in the epsilon-SVR model. It specifies the epsilon-tube
|       within which no penalty is associated in the training loss function
|       with points predicted within a distance epsilon from the actual
|       value. Must be non-negative.
|
|  shrinking : bool, default=True
|      Whether to use the shrinking heuristic.
|      See the :ref:`User Guide <shrinking_svm>`.
|
|  cache_size : float, default=200
|      Specify the size of the kernel cache (in MB).
|
|  verbose : bool, default=False
|      Enable verbose output. Note that this setting takes advantage of a
|      per-process runtime setting in libsvm that, if enabled, may not work
|      properly in a multithreaded context.
|
|  max_iter : int, default=-1
|      Hard limit on iterations within solver, or -1 for no limit.
|
|  Attributes
|  ----------
|  coef_ : ndarray or sparse array/matrix of shape (1, n_features)
|      Weights assigned to the features (coefficients in the primal
|      problem). This is only available in the case of a linear kernel.
|
|      `coef_` is readonly property derived from `dual_coef_` and
|      `support_vectors_`.
|
|  dual_coef_ : ndarray or sparse array/matrix of shape (1, n_SV)
|      Coefficients of the support vector in the decision function.
|      If `X` is sparse, these will also be sparse.
|
|  fit_status_ : int
|      0 if correctly fitted, 1 otherwise (will raise warning)
|
|  intercept_ : ndarray of shape (1,)
|      Constants in decision function.
|
|  n_features_in_ : int
```

```
 |      Number of features seen during :term:`fit`.
 |
 |      .. versionadded:: 0.24
 |
 |  feature_names_in_ : ndarray of shape (`n_features_in_`,)
 |      Names of features seen during :term:`fit`. Defined only when `X`
 |      has feature names that are all strings.
 |
 |      .. versionadded:: 1.0
 |
 |  n_iter_ : int
 |      Number of iterations run by the optimization routine to fit the mode
l.
 |
 |      .. versionadded:: 1.1
 |
 |  n_support_ : ndarray of shape (1,), dtype=int32
 |      Number of support vectors.
 |
 |  shape_fit_ : tuple of int of shape (n_dimensions_of_X,)
 |      Array dimensions of training vector ``X``.
 |
 |  support_ : ndarray of shape (n_SV,)
 |      Indices of support vectors.
 |
 |  support_vectors_ : ndarray or sparse array/matrix of shape (n_SV, n_feat
ures)
 |      Support vectors. If `X` is sparse, these will also be sparse.
 |
 |  See Also
 |  --------
 |  NuSVR : Support Vector Machine for regression implemented using libsvm
 |      using a parameter to control the number of support vectors.
 |
 |  LinearSVR : Scalable Linear Support Vector Machine for regression
 |      implemented using liblinear.
 |
 |  References
 |  ----------
 |  .. [1] `LIBSVM: A Library for Support Vector Machines
 |      <http://www.csie.ntu.edu.tw/~cjlin/papers/libsvm.pdf>`_
 |
 |  .. [2] `Platt, John (1999). "Probabilistic Outputs for Support Vector
 |      Machines and Comparisons to Regularized Likelihood Methods"
 |      <https://citeseerx.ist.psu.edu/doc_view/pid/42e5ed832d4310ce4378c44d
05570439df28a393>`_
 |
 |  Examples
 |  --------
 |  >>> from sklearn.svm import SVR
 |  >>> from sklearn.pipeline import make_pipeline
 |  >>> from sklearn.preprocessing import StandardScaler
 |  >>> import numpy as np
 |  >>> n_samples, n_features = 10, 5
 |  >>> rng = np.random.RandomState(0)
 |  >>> y = rng.randn(n_samples)
```

```
 |  >>> X = rng.randn(n_samples, n_features)
 |  >>> regr = make_pipeline(StandardScaler(), SVR(C=1.0, epsilon=0.2))
 |  >>> regr.fit(X, y)
 |  Pipeline(steps=[('standardscaler', StandardScaler()),
 |                  ('svr', SVR(epsilon=0.2))])
 |
 |  Method resolution order:
 |      SVR
 |      sklearn.base.RegressorMixin
 |      sklearn.svm._base.BaseLibSVM
 |      sklearn.base.BaseEstimator
 |      sklearn.utils._repr_html.base.ReprHTMLMixin
 |      sklearn.utils._repr_html.base._HTMLDocumentationLinkMixin
 |      sklearn.utils._metadata_requests._MetadataRequester
 |      builtins.object
 |
 |  Methods defined here:
 |
 |  __init__(
 |      self,
 |      *,
 |      kernel='rbf',
 |      degree=3,
 |      gamma='scale',
 |      coef0=0.0,
 |      tol=0.001,
 |      C=1.0,
 |      epsilon=0.1,
 |      shrinking=True,
 |      cache_size=200,
 |      verbose=False,
 |      max_iter=-1
 |  )
 |      Initialize self.  See help(type(self)) for accurate signature.
 |
 |  set_fit_request(
 |      self: sklearn.svm._classes.SVR,
 |      *,
 |      sample_weight: bool | None | str = '$UNCHANGED$'
 |  ) -> sklearn.svm._classes.SVR from sklearn.utils._metadata_requests.Requ
estMethod.__get__.<locals>
 |      Configure whether metadata should be requested to be passed to the `
`fit`` method.
 |
 |      Note that this method is only relevant when this estimator is used a
s a
 |      sub-estimator within a :term:`meta-estimator` and metadata routing i
s enabled
 |      with ``enable_metadata_routing=True`` (see :func:`sklearn.set_config
`).
 |      Please check the :ref:`User Guide <metadata_routing>` on how the rou
ting
 |      mechanism works.
 |
 |      The options for each parameter are:
 |
```

```
 |      - ``True``: metadata is requested, and passed to ``fit`` if provide
d. The request is ignored if metadata is not provided.
 |
 |      - ``False``: metadata is not requested and the meta-estimator will n
ot pass it to ``fit``.
 |
 |      - ``None``: metadata is not requested, and the meta-estimator will r
aise an error if the user provides it.
 |
 |      - ``str``: metadata should be passed to the meta-estimator with this
given alias instead of the original name.
 |
 |      The default (``sklearn.utils.metadata_routing.UNCHANGED``) retains t
he
 |      existing request. This allows you to change the request for some
 |      parameters and not others.
 |
 |      .. versionadded:: 1.3
 |
 |      Parameters
 |      ----------
 |      sample_weight : str, True, False, or None,                     defau
lt=sklearn.utils.metadata_routing.UNCHANGED
 |          Metadata routing for ``sample_weight`` parameter in ``fit``.
 |
 |      Returns
 |      -------
 |      self : object
 |          The updated object.
 |
 |  set_score_request(
 |      self: sklearn.svm._classes.SVR,
 |      *,
 |      sample_weight: bool | None | str = '$UNCHANGED$'
 |  ) -> sklearn.svm._classes.SVR from sklearn.utils._metadata_requests.Requ
estMethod.__get__.<locals>
 |      Configure whether metadata should be requested to be passed to the `
`score`` method.
 |
 |      Note that this method is only relevant when this estimator is used a
s a
 |      sub-estimator within a :term:`meta-estimator` and metadata routing i
s enabled
 |      with ``enable_metadata_routing=True`` (see :func:`sklearn.set_config
`).
 |      Please check the :ref:`User Guide <metadata_routing>` on how the rou
ting
 |      mechanism works.
 |
 |      The options for each parameter are:
 |
 |      - ``True``: metadata is requested, and passed to ``score`` if provid
ed. The request is ignored if metadata is not provided.
 |
 |      - ``False``: metadata is not requested and the meta-estimator will n
ot pass it to ``score``.
```

```
 |
 |      - ``None``: metadata is not requested, and the meta-estimator will r
aise an error if the user provides it.
 |
 |      - ``str``: metadata should be passed to the meta-estimator with this
given alias instead of the original name.
 |
 |      The default (``sklearn.utils.metadata_routing.UNCHANGED``) retains t
he
 |      existing request. This allows you to change the request for some
 |      parameters and not others.
 |
 |      .. versionadded:: 1.3
 |
 |      Parameters
 |      ----------
 |      sample_weight : str, True, False, or None,                     defau
lt=sklearn.utils.metadata_routing.UNCHANGED
 |          Metadata routing for ``sample_weight`` parameter in ``score``.
 |
 |      Returns
 |      -------
 |      self : object
 |          The updated object.
 |
 |  ----------------------------------------------------------------------
 |  Data and other attributes defined here:
 |
 |  __abstractmethods__ = frozenset()
 |
 |  unused_param = 'random_state'
 |
 |  ----------------------------------------------------------------------
 |  Methods inherited from sklearn.base.RegressorMixin:
 |
 |  __sklearn_tags__(self)
 |
 |  score(self, X, y, sample_weight=None)
 |      Return :ref:`coefficient of determination <r2_score>` on test data.
 |
 |      The coefficient of determination, :math:`R^2`, is defined as
 |      :math:`(1 - \frac{u}{v})`, where :math:`u` is the residual
 |      sum of squares ``((y_true - y_pred)** 2).sum()`` and :math:`v`
 |      is the total sum of squares ``((y_true - y_true.mean()) ** 2).sum()`
`.
 |      The best possible score is 1.0 and it can be negative (because the
 |      model can be arbitrarily worse). A constant model that always predic
ts
 |      the expected value of `y`, disregarding the input features, would ge
t
 |      a :math:`R^2` score of 0.0.
 |
 |      Parameters
 |      ----------
 |      X : array-like of shape (n_samples, n_features)
 |          Test samples. For some estimators this may be a precomputed
```

```
 |          kernel matrix or a list of generic objects instead with shape
 |          ``(n_samples, n_samples_fitted)``, where ``n_samples_fitted``
 |          is the number of samples used in the fitting for the estimator.
 |
 |      y : array-like of shape (n_samples,) or (n_samples, n_outputs)
 |          True values for `X`.
 |
 |      sample_weight : array-like of shape (n_samples,), default=None
 |          Sample weights.
 |
 |      Returns
 |      -------
 |      score : float
 |          :math:`R^2` of ``self.predict(X)`` w.r.t. `y`.
 |
 |      Notes
 |      -----
 |      The :math:`R^2` score used when calling ``score`` on a regressor use
s
 |      ``multioutput='uniform_average'`` from version 0.23 to keep consiste
nt
 |      with default value of :func:`~sklearn.metrics.r2_score`.
 |      This influences the ``score`` method of all the multioutput
 |      regressors (except for
 |      :class:`~sklearn.multioutput.MultiOutputRegressor`).
 |
 |  ----------------------------------------------------------------------
 |  Data descriptors inherited from sklearn.base.RegressorMixin:
 |
 |  __dict__
 |      dictionary for instance variables
 |
 |  __weakref__
 |      list of weak references to the object
 |
 |  ----------------------------------------------------------------------
 |  Methods inherited from sklearn.svm._base.BaseLibSVM:
 |
 |  fit(self, X, y, sample_weight=None)
 |      Fit the SVM model according to the given training data.
 |
 |      Parameters
 |      ----------
 |      X : {array-like, sparse matrix} of shape (n_samples, n_features)
or (n_samples, n_samples)
 |          Training vectors, where `n_samples` is the number of samples
 |          and `n_features` is the number of features.
 |          For kernel="precomputed", the expected shape of X is
 |          (n_samples, n_samples).
 |
 |      y : array-like of shape (n_samples,)
 |          Target values (class labels in classification, real numbers in
 |          regression).
 |
 |      sample_weight : array-like of shape (n_samples,), default=None
 |          Per-sample weights. Rescale C per sample. Higher weights
```

```
 |          force the classifier to put more emphasis on these points.
 |
 |      Returns
 |      -------
 |      self : object
 |          Fitted estimator.
 |
 |      Notes
 |      -----
 |      If X and y are not C-ordered and contiguous arrays of np.float64 and
 |      X is not a sparse CSR format, X and/or y may be copied.
 |
 |      If X is a dense array, then the other methods will not support spars
e
 |      matrices as input.
 |
 |  predict(self, X)
 |      Perform regression on samples in X.
 |
 |      For a one-class model, +1 (inlier) or -1 (outlier) is returned.
 |
 |      Parameters
 |      ----------
 |      X : {array-like, sparse matrix} of shape (n_samples, n_features)
 |          For kernel="precomputed", the expected shape of X is
 |          (n_samples_test, n_samples_train).
 |
 |      Returns
 |      -------
 |      y_pred : ndarray of shape (n_samples,)
 |          The predicted values.
 |
 |  ----------------------------------------------------------------------
 |  Readonly properties inherited from sklearn.svm._base.BaseLibSVM:
 |
 |  coef_
 |      Weights assigned to the features when `kernel="linear"`.
 |
 |      Returns
 |      -------
 |      ndarray of shape (n_features, n_classes)
 |
 |  n_support_
 |      Number of support vectors for each class.
 |
 |  ----------------------------------------------------------------------
 |  Methods inherited from sklearn.base.BaseEstimator:
 |
 |  __dir__(self)
 |      Default dir() implementation.
 |
 |  __getstate__(self)
 |      Helper for pickle.
 |
 |  __repr__(self, N_CHAR_MAX=700)
 |      Return repr(self).
```

```
 |
 |  __setstate__(self, state)
 |
 |  __sklearn_clone__(self)
 |
 |  get_params(self, deep=True)
 |      Get parameters for this estimator.
 |
 |      Parameters
 |      ----------
 |      deep : bool, default=True
 |          If True, will return the parameters for this estimator and
 |          contained subobjects that are estimators.
 |
 |      Returns
 |      -------
 |      params : dict
 |          Parameter names mapped to their values.
 |
 |  set_params(self, **params)
 |      Set the parameters of this estimator.
 |
 |      The method works on simple estimators as well as on nested objects
 |      (such as :class:`~sklearn.pipeline.Pipeline`). The latter have
 |      parameters of the form ``<component>__<parameter>`` so that it's
 |      possible to update each component of a nested object.
 |
 |      Parameters
 |      ----------
 |      **params : dict
 |          Estimator parameters.
 |
 |      Returns
 |      -------
 |      self : estimator instance
 |          Estimator instance.
 |
 |  ----------------------------------------------------------------------
 |  Methods inherited from sklearn.utils._metadata_requests._MetadataRequest
er:
 |
 |  get_metadata_routing(self)
 |      Get metadata routing of this object.
 |
 |      Please check :ref:`User Guide <metadata_routing>` on how the routing
 |      mechanism works.
 |
 |      Returns
 |      -------
 |      routing : MetadataRequest
 |          A :class:`~sklearn.utils.metadata_routing.MetadataRequest` encap
sulating
 |          routing information.
 |
 |  ----------------------------------------------------------------------
 |  Class methods inherited from sklearn.utils._metadata_requests._MetadataR
```

```
equester:
 |
 |  __init_subclass__(**kwargs)
 |      Set the ``set_{method}_request`` methods.
 |
 |      This uses PEP-487 [1]_ to set the ``set_{method}_request`` methods.
It
 |      looks for the information available in the set default values which
are
 |      set using ``__metadata_request__*`` class attributes, or inferred
 |      from method signatures.
 |
 |      The ``__metadata_request__*`` class attributes are used when a metho
d
 |      does not explicitly accept a metadata through its arguments or if th
e
 |      developer would like to specify a request value for those metadata
 |      which are different from the default ``None``.
 |
 |      References
 |      ----------
 |      .. [1] https://www.python.org/dev/peps/pep-0487
```

In [6]:
```
import matplotlib.pyplot as plt
import matplotlib


plt.figure(figsize=(8,6))

plt.subplot(2,2,1)
plt.scatter(X,y,label='training data')
plt.plot(np.linspace(0, 1, 100),true_fun(np.linspace(0, 1, 100)),label='true
reg = SVR(gamma = 1000000, C = 1e4)
reg.fit(X[:, np.newaxis],y)
y_new = reg.predict(X_new[:, np.newaxis])
plt.plot(X_new,y_new,'r',label='prediction')
plt.xlabel('x')
plt.ylabel('y')
plt.title('gamma = 1e6')
plt.legend()

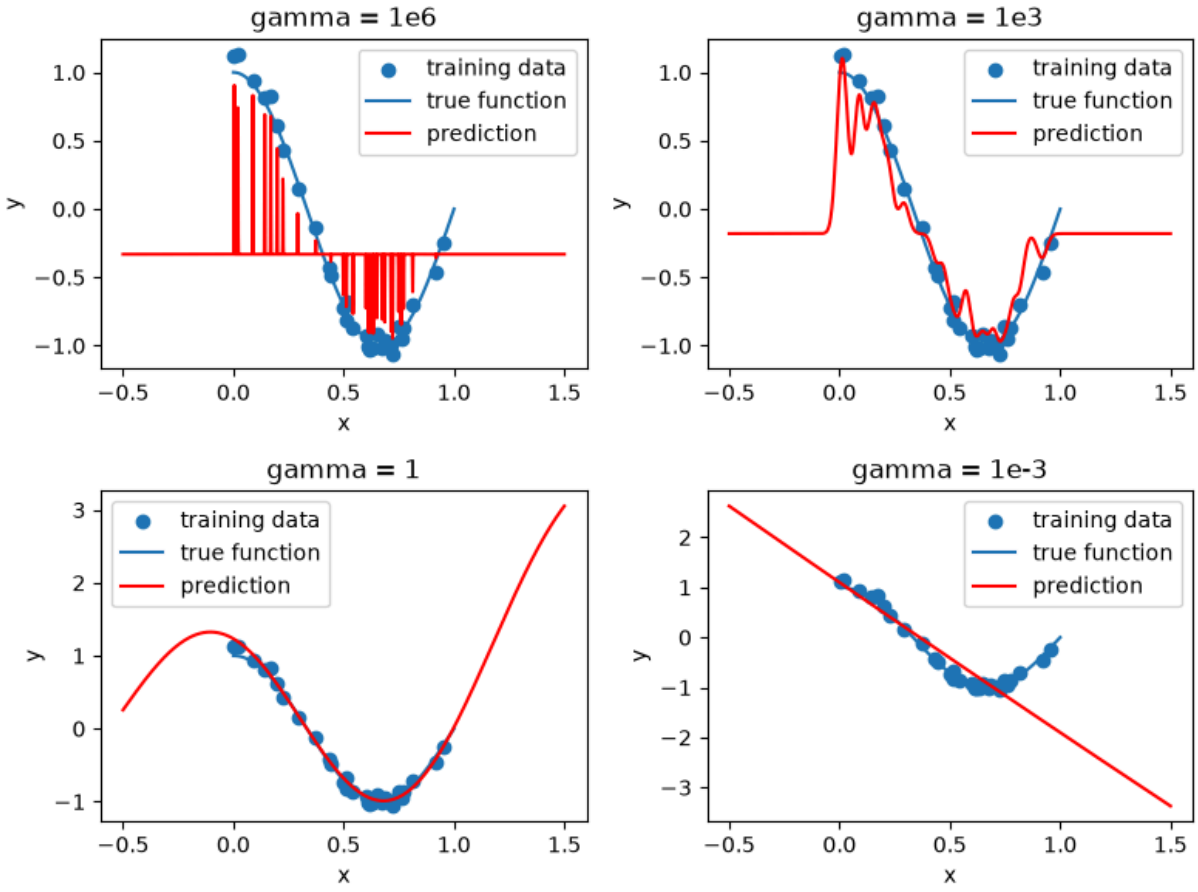
plt.subplot(2,2,2)
plt.scatter(X,y,label='training data')
plt.plot(np.linspace(0, 1, 100),true_fun(np.linspace(0, 1, 100)),label='true
reg = SVR(gamma = 1000, C = 1e4)
reg.fit(X[:, np.newaxis],y)
y_new = reg.predict(X_new[:, np.newaxis])
plt.plot(X_new,y_new,'r',label='prediction')
plt.xlabel('x')
plt.ylabel('y')
plt.title('gamma = 1e3')
plt.legend()

plt.subplot(2,2,3)
plt.scatter(X,y,label='training data')
```

```
plt.plot(np.linspace(0, 1, 100),true_fun(np.linspace(0, 1, 100)),label='true
reg = SVR(gamma = 1, C = 1e4)
reg.fit(X[:, np.newaxis],y)
y_new = reg.predict(X_new[:, np.newaxis])
plt.plot(X_new,y_new,'r',label='prediction')
plt.xlabel('x')
plt.ylabel('y')
plt.title('gamma = 1')
plt.legend()


plt.subplot(2,2,4)
plt.scatter(X,y,label='training data')
plt.plot(np.linspace(0, 1, 100),true_fun(np.linspace(0, 1, 100)),label='true
reg = SVR(gamma = 0.001, C = 1e4)
reg.fit(X[:, np.newaxis],y)
y_new = reg.predict(X_new[:, np.newaxis])
plt.plot(X_new,y_new,'r',label='prediction')
plt.xlabel('x')
plt.ylabel('y')
plt.title('gamma = 1e-3')
plt.legend()

plt.tight_layout()
plt.savefig('../figures/SVM_reg.png',dpi=300)
plt.show()
```



SVM in classification

In [7]:
```
from sklearn.datasets import make_moons
import numpy as np
from sklearn.svm import SVC
from sklearn.calibration import CalibratedClassifierCV

# create the data
X,y = make_moons(noise=0.2, random_state=1,n_samples=200)
# set the hyperparameters
# NOTE: sklearn's SVC does not return predicted probabilities, only hard pre
# If you need an SVM with predicted probabilities, use CalibratedClassifierC
clf = CalibratedClassifierCV(SVC(gamma = 1, C = 1), ensemble=False) # the de
# fit the model
clf.fit(X,y)
# predict new data
#y_new = clf.predict(X_new)
# predict probabilities
#y_new = clf.predict_proba(X_new)
```

Out[7]:

▸ **CalibratedClassifierCV** ⓘ ⓘ

▸ **estimator: SVC**

▸ SVC ⓘ

In [8]:
```
help(SVC)
```

```
Help on class SVC in module sklearn.svm._classes:

class SVC(sklearn.svm._base.BaseSVC)
 |  SVC(
 |      *,
 |      C=1.0,
 |      kernel='rbf',
 |      degree=3,
 |      gamma='scale',
 |      coef0=0.0,
 |      shrinking=True,
 |      probability='deprecated',
 |      tol=0.001,
 |      cache_size=200,
 |      class_weight=None,
 |      verbose=False,
 |      max_iter=-1,
 |      decision_function_shape='ovr',
 |      break_ties=False,
 |      random_state=None
 |  )
 |
 |  C-Support Vector Classification.
 |
 |  The implementation is based on libsvm. The fit time scales at least
 |  quadratically with the number of samples and may be impractical
 |  beyond tens of thousands of samples. For large datasets
 |  consider using :class:`~sklearn.svm.LinearSVC` or
 |  :class:`~sklearn.linear_model.SGDClassifier` instead, possibly after a
 |  :class:`~sklearn.kernel_approximation.Nystroem` transformer or
 |  other :ref:`kernel_approximation`.
 |
 |  The multiclass support is handled according to a one-vs-one scheme.
 |
 |  For details on the precise mathematical formulation of the provided
 |  kernel functions and how `gamma`, `coef0` and `degree` affect each
 |  other, see the corresponding section in the narrative documentation:
 |  :ref:`svm_kernels`.
 |
 |  To learn how to tune SVC's hyperparameters, see the following example:
 |  :ref:`sphx_glr_auto_examples_model_selection_plot_nested_cross_validatio
n_iris.py`
 |
 |  Read more in the :ref:`User Guide <svm_classification>`.
 |
 |  Parameters
 |  ----------
 |  C : float, default=1.0
 |      Regularization parameter. The strength of the regularization is
 |      inversely proportional to C. Must be strictly positive. The penalty
 |      is a squared l2 penalty. For an intuitive visualization of the effec
ts
 |      of scaling the regularization parameter C, see
 |      :ref:`sphx_glr_auto_examples_svm_plot_svm_scale_c.py`.
 |
 |  kernel : {'linear', 'poly', 'rbf', 'sigmoid', 'precomputed'} or callabl
```

```
e,          default='rbf'
 |      Specifies the kernel type to be used in the algorithm. If
 |      none is given, 'rbf' will be used. If a callable is given it is used
to
 |      pre-compute the kernel matrix from data matrices; that matrix should
be
 |      an array of shape ``(n_samples, n_samples)``. For an intuitive
 |      visualization of different kernel types see
 |      :ref:`sphx_glr_auto_examples_svm_plot_svm_kernels.py`.
 |
 |  degree : int, default=3
 |      Degree of the polynomial kernel function ('poly').
 |      Must be non-negative. Ignored by all other kernels.
 |
 |  gamma : {'scale', 'auto'} or float, default='scale'
 |      Kernel coefficient for 'rbf', 'poly' and 'sigmoid'.
 |
 |      - if ``gamma='scale'`` (default) is passed then it uses
 |        1 / (n_features * X.var()) as value of gamma,
 |      - if 'auto', uses 1 / n_features
 |      - if float, must be non-negative.
 |
 |      .. versionchanged:: 0.22
 |         The default value of ``gamma`` changed from 'auto' to 'scale'.
 |
 |  coef0 : float, default=0.0
 |      Independent term in kernel function.
 |      It is only significant in 'poly' and 'sigmoid'.
 |
 |  shrinking : bool, default=True
 |      Whether to use the shrinking heuristic.
 |      See the :ref:`User Guide <shrinking_svm>`.
 |
 |  probability : bool, default=False
 |      Whether to enable probability estimates. This must be enabled prior
 |      to calling `fit`, will slow down that method as it internally uses
 |      5-fold cross-validation, and `predict_proba` may be inconsistent wit
h
 |      `predict`. Read more in the :ref:`User Guide <scores_probabilities>
`.
 |
 |      ..deprecated:: 1.9
 |        The `probability` parameter is deprecated and will be removed in
1.11.
 |        Use `CalibratedClassifierCV(SVC(), ensemble=False)` instead of
 |        `SVC(probability=True)`.
 |
 |  tol : float, default=1e-3
 |      Tolerance for stopping criterion.
 |
 |  cache_size : float, default=200
 |      Specify the size of the kernel cache (in MB).
 |
 |  class_weight : dict or 'balanced', default=None
 |      Set the parameter C of class i to class_weight[i]*C for
 |      SVC. If not given, all classes are supposed to have
```

```
 |      weight one.
 |      The "balanced" mode uses the values of y to automatically adjust
 |      weights inversely proportional to class frequencies in the input dat
a
 |      as ``n_samples / (n_classes * np.bincount(y))``.
 |
 |  verbose : bool, default=False
 |      Enable verbose output. Note that this setting takes advantage of a
 |      per-process runtime setting in libsvm that, if enabled, may not work
 |      properly in a multithreaded context.
 |
 |  max_iter : int, default=-1
 |      Hard limit on iterations within solver, or -1 for no limit.
 |
 |  decision_function_shape : {'ovo', 'ovr'}, default='ovr'
 |      Whether to return a one-vs-rest ('ovr') decision function of shape
 |      (n_samples, n_classes) as all other classifiers, or the original
 |      one-vs-one ('ovo') decision function of libsvm which has shape
 |      (n_samples, n_classes * (n_classes - 1) / 2). However, note that
 |      internally, one-vs-one ('ovo') is always used as a multi-class strat
egy
 |      to train models; an ovr matrix is only constructed from the ovo matr
ix.
 |      The parameter is ignored for binary classification.
 |
 |      .. versionchanged:: 0.19
 |          decision_function_shape is 'ovr' by default.
 |
 |      .. versionadded:: 0.17
 |         *decision_function_shape='ovr'* is recommended.
 |
 |      .. versionchanged:: 0.17
 |         Deprecated *decision_function_shape='ovo' and None*.
 |
 |  break_ties : bool, default=False
 |      If true, ``decision_function_shape='ovr'``, and number of classes >
2,
 |      :term:`predict` will break ties according to the confidence values o
f
 |      :term:`decision_function`; otherwise the first class among the tied
 |      classes is returned. Please note that breaking ties comes at a
 |      relatively high computational cost compared to a simple predict. See
 |      :ref:`sphx_glr_auto_examples_svm_plot_svm_tie_breaking.py` for an
 |      example of its usage with ``decision_function_shape='ovr'``.
 |
 |      .. versionadded:: 0.22
 |
 |  random_state : int, RandomState instance or None, default=None
 |      Controls the pseudo random number generation for shuffling the data
for
 |      probability estimates. Ignored when `probability` is False.
 |      Pass an int for reproducible output across multiple function calls.
 |      See :term:`Glossary <random_state>`.
 |
 |  Attributes
 |  ----------
```

```
 |  class_weight_ : ndarray of shape (n_classes,)
 |      Multipliers of parameter C for each class.
 |      Computed based on the ``class_weight`` parameter.
 |
 |  classes_ : ndarray of shape (n_classes,)
 |      The classes labels.
 |
 |  coef_ : ndarray or sparse array/matrix            of shape (n_classes *
(n_classes - 1) / 2, n_features)
 |      Weights assigned to the features (coefficients in the primal
 |      problem). This is only available in the case of a linear kernel.
 |
 |      `coef_` is a readonly property derived from `dual_coef_` and
 |      `support_vectors_`.
 |
 |  dual_coef_ : ndarray or sparse array/matrix of shape (n_classes -1, n_S
V)
 |      Dual coefficients of the support vector in the decision
 |      function (see :ref:`sgd_mathematical_formulation`), multiplied by
 |      their targets.
 |      For multiclass, coefficient for all 1-vs-1 classifiers.
 |      The layout of the coefficients in the multiclass case is somewhat
 |      non-trivial. See the :ref:`multi-class section of the User Guide
 |      <svm_multi_class>` for details.
 |      If `X` is sparse, these will also be sparse.
 |
 |  fit_status_ : int
 |      0 if correctly fitted, 1 otherwise (will raise warning)
 |
 |  intercept_ : ndarray of shape (n_classes * (n_classes - 1) / 2,)
 |      Constants in decision function.
 |
 |  n_features_in_ : int
 |      Number of features seen during :term:`fit`.
 |
 |      .. versionadded:: 0.24
 |
 |  feature_names_in_ : ndarray of shape (`n_features_in_`,)
 |      Names of features seen during :term:`fit`. Defined only when `X`
 |      has feature names that are all strings.
 |
 |      .. versionadded:: 1.0
 |
 |  n_iter_ : ndarray of shape (n_classes * (n_classes - 1) // 2,)
 |      Number of iterations run by the optimization routine to fit the mode
l.
 |      The shape of this attribute depends on the number of models optimize
d
 |      which in turn depends on the number of classes.
 |
 |      .. versionadded:: 1.1
 |
 |  support_ : ndarray of shape (n_SV)
 |      Indices of support vectors.
 |
 |  support_vectors_ : ndarray or sparse array/matrix of shape (n_SV, n_feat
```

```
ures)
 |      Support vectors. An empty array if kernel is precomputed.
 |      If `X` is sparse, these will also be sparse.
 |
 |  n_support_ : ndarray of shape (n_classes,), dtype=int32
 |      Number of support vectors for each class.
 |
 |  probA_ : ndarray of shape (n_classes * (n_classes - 1) / 2)
 |      If `probability=True`, it corresponds to the parameters learned in
 |      Platt scaling to produce probability estimates from decision values.
 |      If `probability=False`, it's an empty array. Platt scaling uses the
 |      logistic function
 |
 |  probB_ : ndarray of shape (n_classes * (n_classes - 1) / 2)
 |      If `probability=True`, it corresponds to the parameters learned in
 |      Platt scaling. Platt scaling uses the logistic function
 |      ``1 / (1 + exp(decision_value * probA_ + probB_))``
 |      where ``probA_`` and ``probB_`` are learned from the dataset [2]_. F
or
 |      more information on the multiclass case and training procedure see
 |      section 8 of [1]_.
 |
 |      .. deprecated:: 1.9
 |          The attributes `probA_` and `probB_` are deprecated in version
1.9 and will
 |          be removed in 1.11.
 |
 |  shape_fit_ : tuple of int of shape (n_dimensions_of_X,)
 |      Array dimensions of training vector ``X``.
 |
 |  See Also
 |  --------
 |  SVR : Support Vector Machine for Regression implemented using libsvm.
 |
 |  LinearSVC : Scalable Linear Support Vector Machine for classification
 |      implemented using liblinear. Check the See Also section of
 |      LinearSVC for more comparison element.
 |
 |  References
 |  ----------
 |  .. [1] `LIBSVM: A Library for Support Vector Machines
 |      <http://www.csie.ntu.edu.tw/~cjlin/papers/libsvm.pdf>`_
 |
 |  .. [2] `Platt, John (1999). "Probabilistic Outputs for Support Vector
 |      Machines and Comparisons to Regularized Likelihood Methods"
 |      <https://citeseerx.ist.psu.edu/doc_view/pid/42e5ed832d4310ce4378c44d
05570439df28a393>`_
 |
 |  Examples
 |  --------
 |  >>> import numpy as np
 |  >>> from sklearn.pipeline import make_pipeline
 |  >>> from sklearn.preprocessing import StandardScaler
 |  >>> X = np.array([[-1, -1], [-2, -1], [1, 1], [2, 1]])
 |  >>> y = np.array([1, 1, 2, 2])
 |  >>> from sklearn.svm import SVC
```

```
 |  >>> clf = make_pipeline(StandardScaler(), SVC(gamma='auto'))
 |  >>> clf.fit(X, y)
 |  Pipeline(steps=[('standardscaler', StandardScaler()),
 |                  ('svc', SVC(gamma='auto'))])
 |
 |  >>> print(clf.predict([[-0.8, -1]]))
 |  [1]
 |
 |  For a comparison of the SVC with other classifiers see:
 |  :ref:`sphx_glr_auto_examples_classification_plot_classification_probabil
ity.py`.
 |
 |  Method resolution order:
 |      SVC
 |      sklearn.svm._base.BaseSVC
 |      sklearn.base.ClassifierMixin
 |      sklearn.svm._base.BaseLibSVM
 |      sklearn.base.BaseEstimator
 |      sklearn.utils._repr_html.base.ReprHTMLMixin
 |      sklearn.utils._repr_html.base._HTMLDocumentationLinkMixin
 |      sklearn.utils._metadata_requests._MetadataRequester
 |      builtins.object
 |
 |  Methods defined here:
 |
 |  __init__(
 |      self,
 |      *,
 |      C=1.0,
 |      kernel='rbf',
 |      degree=3,
 |      gamma='scale',
 |      coef0=0.0,
 |      shrinking=True,
 |      probability='deprecated',
 |      tol=0.001,
 |      cache_size=200,
 |      class_weight=None,
 |      verbose=False,
 |      max_iter=-1,
 |      decision_function_shape='ovr',
 |      break_ties=False,
 |      random_state=None
 |  )
 |      Initialize self.  See help(type(self)) for accurate signature.
 |
 |  set_fit_request(
 |      self: sklearn.svm._classes.SVC,
 |      *,
 |      sample_weight: bool | None | str = '$UNCHANGED$'
 |  ) -> sklearn.svm._classes.SVC from sklearn.utils._metadata_requests.Requ
estMethod.__get__.<locals>
 |      Configure whether metadata should be requested to be passed to the `
`fit`` method.
 |
 |      Note that this method is only relevant when this estimator is used a
```

```
s a
 |      sub-estimator within a :term:`meta-estimator` and metadata routing i
s enabled
 |      with ``enable_metadata_routing=True`` (see :func:`sklearn.set_config
`).
 |      Please check the :ref:`User Guide <metadata_routing>` on how the rou
ting
 |      mechanism works.
 |
 |      The options for each parameter are:
 |
 |      - ``True``: metadata is requested, and passed to ``fit`` if provide
d. The request is ignored if metadata is not provided.
 |
 |      - ``False``: metadata is not requested and the meta-estimator will n
ot pass it to ``fit``.
 |
 |      - ``None``: metadata is not requested, and the meta-estimator will r
aise an error if the user provides it.
 |
 |      - ``str``: metadata should be passed to the meta-estimator with this
given alias instead of the original name.
 |
 |      The default (``sklearn.utils.metadata_routing.UNCHANGED``) retains t
he
 |      existing request. This allows you to change the request for some
 |      parameters and not others.
 |
 |      .. versionadded:: 1.3
 |
 |      Parameters
 |      ----------
 |      sample_weight : str, True, False, or None,                     defau
lt=sklearn.utils.metadata_routing.UNCHANGED
 |          Metadata routing for ``sample_weight`` parameter in ``fit``.
 |
 |      Returns
 |      -------
 |      self : object
 |          The updated object.
 |
 |  set_score_request(
 |      self: sklearn.svm._classes.SVC,
 |      *,
 |      sample_weight: bool | None | str = '$UNCHANGED$'
 |  ) -> sklearn.svm._classes.SVC from sklearn.utils._metadata_requests.Requ
estMethod.__get__.<locals>
 |      Configure whether metadata should be requested to be passed to the `
`score`` method.
 |
 |      Note that this method is only relevant when this estimator is used a
s a
 |      sub-estimator within a :term:`meta-estimator` and metadata routing i
s enabled
 |      with ``enable_metadata_routing=True`` (see :func:`sklearn.set_config
`).
```

```
 |      Please check the :ref:`User Guide <metadata_routing>` on how the rou
ting
 |      mechanism works.
 |
 |      The options for each parameter are:
 |
 |      - ``True``: metadata is requested, and passed to ``score`` if provid
ed. The request is ignored if metadata is not provided.
 |
 |      - ``False``: metadata is not requested and the meta-estimator will n
ot pass it to ``score``.
 |
 |      - ``None``: metadata is not requested, and the meta-estimator will r
aise an error if the user provides it.
 |
 |      - ``str``: metadata should be passed to the meta-estimator with this
given alias instead of the original name.
 |
 |      The default (``sklearn.utils.metadata_routing.UNCHANGED``) retains t
he
 |      existing request. This allows you to change the request for some
 |      parameters and not others.
 |
 |      .. versionadded:: 1.3
 |
 |      Parameters
 |      ----------
 |      sample_weight : str, True, False, or None,                     defau
lt=sklearn.utils.metadata_routing.UNCHANGED
 |          Metadata routing for ``sample_weight`` parameter in ``score``.
 |
 |      Returns
 |      -------
 |      self : object
 |          The updated object.
 |
 |  ----------------------------------------------------------------------
 |  Data and other attributes defined here:
 |
 |  __abstractmethods__ = frozenset()
 |
 |  ----------------------------------------------------------------------
 |  Methods inherited from sklearn.svm._base.BaseSVC:
 |
 |  __sklearn_tags__(self)
 |
 |  decision_function(self, X)
 |      Evaluate the decision function for the samples in X.
 |
 |      Parameters
 |      ----------
 |      X : array-like of shape (n_samples, n_features)
 |          The input samples.
 |
 |      Returns
 |      -------
```

```
 |      X : ndarray of shape (n_samples, n_classes * (n_classes-1) / 2)
 |          Returns the decision function of the sample for each class
 |          in the model.
 |          If decision_function_shape='ovr', the shape is (n_samples,
 |          n_classes).
 |
 |      Notes
 |      -----
 |      If decision_function_shape='ovo', the function values are proportion
al
 |      to the distance of the samples X to the separating hyperplane. If th
e
 |      exact distances are required, divide the function values by the norm
of
 |      the weight vector (``coef_``). See also `this question
 |      <https://stats.stackexchange.com/questions/14876/
 |      interpreting-distance-from-hyperplane-in-svm>`_ for further details.
 |      If decision_function_shape='ovr', the decision function is a monoton
ic
 |      transformation of ovo decision function.
 |
 |  predict(self, X)
 |      Perform classification on samples in X.
 |
 |      For a one-class model, +1 or -1 is returned.
 |
 |      Parameters
 |      ----------
 |      X : {array-like, sparse matrix} of shape (n_samples, n_features) or
(n_samples_test, n_samples_train)
 |          For kernel="precomputed", the expected shape of X is
 |          (n_samples_test, n_samples_train).
 |
 |      Returns
 |      -------
 |      y_pred : ndarray of shape (n_samples,)
 |          Class labels for samples in X.
 |
 |  predict_log_proba(self, X)
 |      Compute log probabilities of possible outcomes for samples in X.
 |
 |      The model need to have probability information computed at training
 |      time: fit with attribute `probability` set to True.
 |
 |      Parameters
 |      ----------
 |      X : array-like of shape (n_samples, n_features) or
(n_samples_test, n_samples_train)
 |          For kernel="precomputed", the expected shape of X is
 |          (n_samples_test, n_samples_train).
 |
 |      Returns
 |      -------
 |      T : ndarray of shape (n_samples, n_classes)
 |          Returns the log-probabilities of the sample for each class in
 |          the model. The columns correspond to the classes in sorted
```

```
|          order, as they appear in the attribute :term:`classes_`.
|
|      Notes
|      -----
|      The probability model is created using cross validation, so
|      the results can be slightly different than those obtained by
|      predict. Also, it will produce meaningless results on very small
|      datasets.
|
|  predict_proba(self, X)
|      Compute probabilities of possible outcomes for samples in X.
|
|      The model needs to have probability information computed at training
|      time: fit with attribute `probability` set to True.
|
|      Parameters
|      ----------
|      X : array-like of shape (n_samples, n_features)
|          For kernel="precomputed", the expected shape of X is
|          (n_samples_test, n_samples_train).
|
|      Returns
|      -------
|      T : ndarray of shape (n_samples, n_classes)
|          Returns the probability of the sample for each class in
|          the model. The columns correspond to the classes in sorted
|          order, as they appear in the attribute :term:`classes_`.
|
|      Notes
|      -----
|      The probability model is created using cross validation, so
|      the results can be slightly different than those obtained by
|      predict. Also, it will produce meaningless results on very small
|      datasets.
|
|  ----------------------------------------------------------------------
|  Readonly properties inherited from sklearn.svm._base.BaseSVC:
|
|  probA_
|      Parameter learned in Platt scaling when `probability=True`.
|
|      Returns
|      -------
|      ndarray of shape  (n_classes * (n_classes - 1) / 2)
|
|  probB_
|      Parameter learned in Platt scaling when `probability=True`.
|
|      Returns
|      -------
|      ndarray of shape  (n_classes * (n_classes - 1) / 2)
|
|  ----------------------------------------------------------------------
|  Data and other attributes inherited from sklearn.svm._base.BaseSVC:
|
|  unused_param = 'nu'
```

```
    |
    |  ----------------------------------------------------------------------
    |  Methods inherited from sklearn.base.ClassifierMixin:
    |
    |  score(self, X, y, sample_weight=None)
    |      Return :ref:`accuracy <accuracy_score>` on provided data and labels.
    |
    |      In multi-label classification, this is the subset accuracy
    |      which is a harsh metric since you require for each sample that
    |      each label set be correctly predicted.
    |
    |      Parameters
    |      ----------
    |      X : array-like of shape (n_samples, n_features)
    |          Test samples.
    |
    |      y : array-like of shape (n_samples,) or (n_samples, n_outputs)
    |          True labels for `X`.
    |
    |      sample_weight : array-like of shape (n_samples,), default=None
    |          Sample weights.
    |
    |      Returns
    |      -------
    |      score : float
    |          Mean accuracy of ``self.predict(X)`` w.r.t. `y`.
    |
    |  ----------------------------------------------------------------------
    |  Data descriptors inherited from sklearn.base.ClassifierMixin:
    |
    |  __dict__
    |      dictionary for instance variables
    |
    |  __weakref__
    |      list of weak references to the object
    |
    |  ----------------------------------------------------------------------
    |  Methods inherited from sklearn.svm._base.BaseLibSVM:
    |
    |  fit(self, X, y, sample_weight=None)
    |      Fit the SVM model according to the given training data.
    |
    |      Parameters
    |      ----------
    |      X : {array-like, sparse matrix} of shape (n_samples, n_features)
or (n_samples, n_samples)
    |          Training vectors, where `n_samples` is the number of samples
    |          and `n_features` is the number of features.
    |          For kernel="precomputed", the expected shape of X is
    |          (n_samples, n_samples).
    |
    |      y : array-like of shape (n_samples,)
    |          Target values (class labels in classification, real numbers in
    |          regression).
    |
    |      sample_weight : array-like of shape (n_samples,), default=None
```

```
 |          Per-sample weights. Rescale C per sample. Higher weights
 |          force the classifier to put more emphasis on these points.
 |
 |      Returns
 |      -------
 |      self : object
 |          Fitted estimator.
 |
 |      Notes
 |      -----
 |      If X and y are not C-ordered and contiguous arrays of np.float64 and
 |      X is not a sparse CSR format, X and/or y may be copied.
 |
 |      If X is a dense array, then the other methods will not support spars
e
 |      matrices as input.
 |
 |  ----------------------------------------------------------------------
 |  Readonly properties inherited from sklearn.svm._base.BaseLibSVM:
 |
 |  coef_
 |      Weights assigned to the features when `kernel="linear"`.
 |
 |      Returns
 |      -------
 |      ndarray of shape (n_features, n_classes)
 |
 |  n_support_
 |      Number of support vectors for each class.
 |
 |  ----------------------------------------------------------------------
 |  Methods inherited from sklearn.base.BaseEstimator:
 |
 |  __dir__(self)
 |      Default dir() implementation.
 |
 |  __getstate__(self)
 |      Helper for pickle.
 |
 |  __repr__(self, N_CHAR_MAX=700)
 |      Return repr(self).
 |
 |  __setstate__(self, state)
 |
 |  __sklearn_clone__(self)
 |
 |  get_params(self, deep=True)
 |      Get parameters for this estimator.
 |
 |      Parameters
 |      ----------
 |      deep : bool, default=True
 |          If True, will return the parameters for this estimator and
 |          contained subobjects that are estimators.
 |
 |      Returns
```

```
 |      -------
 |      params : dict
 |          Parameter names mapped to their values.
 |
 |  set_params(self, **params)
 |      Set the parameters of this estimator.
 |
 |      The method works on simple estimators as well as on nested objects
 |      (such as :class:`~sklearn.pipeline.Pipeline`). The latter have
 |      parameters of the form ``<component>__<parameter>`` so that it's
 |      possible to update each component of a nested object.
 |
 |      Parameters
 |      ----------
 |      **params : dict
 |          Estimator parameters.
 |
 |      Returns
 |      -------
 |      self : estimator instance
 |          Estimator instance.
 |
 |  ----------------------------------------------------------------------
 |  Methods inherited from sklearn.utils._metadata_requests._MetadataRequest
er:
 |
 |  get_metadata_routing(self)
 |      Get metadata routing of this object.
 |
 |      Please check :ref:`User Guide <metadata_routing>` on how the routing
 |      mechanism works.
 |
 |      Returns
 |      -------
 |      routing : MetadataRequest
 |          A :class:`~sklearn.utils.metadata_routing.MetadataRequest` encap
sulating
 |          routing information.
 |
 |  ----------------------------------------------------------------------
 |  Class methods inherited from sklearn.utils._metadata_requests._MetadataR
equester:
 |
 |  __init_subclass__(**kwargs)
 |      Set the ``set_{method}_request`` methods.
 |
 |      This uses PEP-487 [1]_ to set the ``set_{method}_request`` methods.
It
 |      looks for the information available in the set default values which
are
 |      set using ``__metadata_request__*`` class attributes, or inferred
 |      from method signatures.
 |
 |      The ``__metadata_request__*`` class attributes are used when a metho
d
 |      does not explicitly accept a metadata through its arguments or if th
```

```
e
 |      developer would like to specify a request value for those metadata
 |      which are different from the default ``None``.
 |
 |      References
 |      ----------
 |      .. [1] https://www.python.org/dev/peps/pep-0487
```

In [9]:

```
from sklearn.datasets import make_moons
import numpy as np
import matplotlib.pyplot as plt
import matplotlib
from matplotlib.colors import ListedColormap
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.calibration import CalibratedClassifierCV

X = StandardScaler().fit_transform(X)

h = .02  # step size in the mesh

x_min, x_max = X[:, 0].min() - .5, X[:, 0].max() + .5
y_min, y_max = X[:, 1].min() - .5, X[:, 1].max() + .5
xx, yy = np.meshgrid(np.arange(x_min, x_max, h),
                     np.arange(y_min, y_max, h))

plt.figure(figsize=(8,6))
cm_bright = ListedColormap(['#FF0000', '#0000FF'])
cm = plt.cm.RdBu

plt.subplot(2,2,1)
clf = CalibratedClassifierCV(SVC(gamma = 1e4, C = 100), ensemble=False)
clf.fit(X,y)
Z = clf.predict_proba(np.c_[xx.ravel(), yy.ravel()])[:, 1]
# Put the result into a color plot
Z = Z.reshape(xx.shape)
plt.contourf(xx, yy, Z, cmap=cm, alpha=.8,vmin=0,vmax=1,levels=np.arange(0,1
plt.colorbar(label='predicted prob')
plt.contour(xx, yy, Z, alpha=.8,vmin=0,vmax=1,levels=[0.5],colors=['k'],line
plt.scatter(X[:, 0], X[:, 1], c=y,cmap=cm_bright)
plt.xlabel('feature_1')
plt.ylabel('feature_2')
plt.title('gamma = 1e4')

plt.subplot(2,2,2)
clf = CalibratedClassifierCV(SVC(gamma = 1e2, C = 100), ensemble=False)
clf.fit(X,y)
Z = clf.predict_proba(np.c_[xx.ravel(), yy.ravel()])[:, 1]
# Put the result into a color plot
Z = Z.reshape(xx.shape)
plt.contourf(xx, yy, Z, cmap=cm, alpha=.8,vmin=0,vmax=1,levels=np.arange(0,1
plt.colorbar(label='predicted prob')
plt.contour(xx, yy, Z, alpha=.8,vmin=0,vmax=1,levels=[0.5],colors=['k'],line
plt.scatter(X[:, 0], X[:, 1], c=y,cmap=cm_bright)
plt.xlabel('feature_1')
```

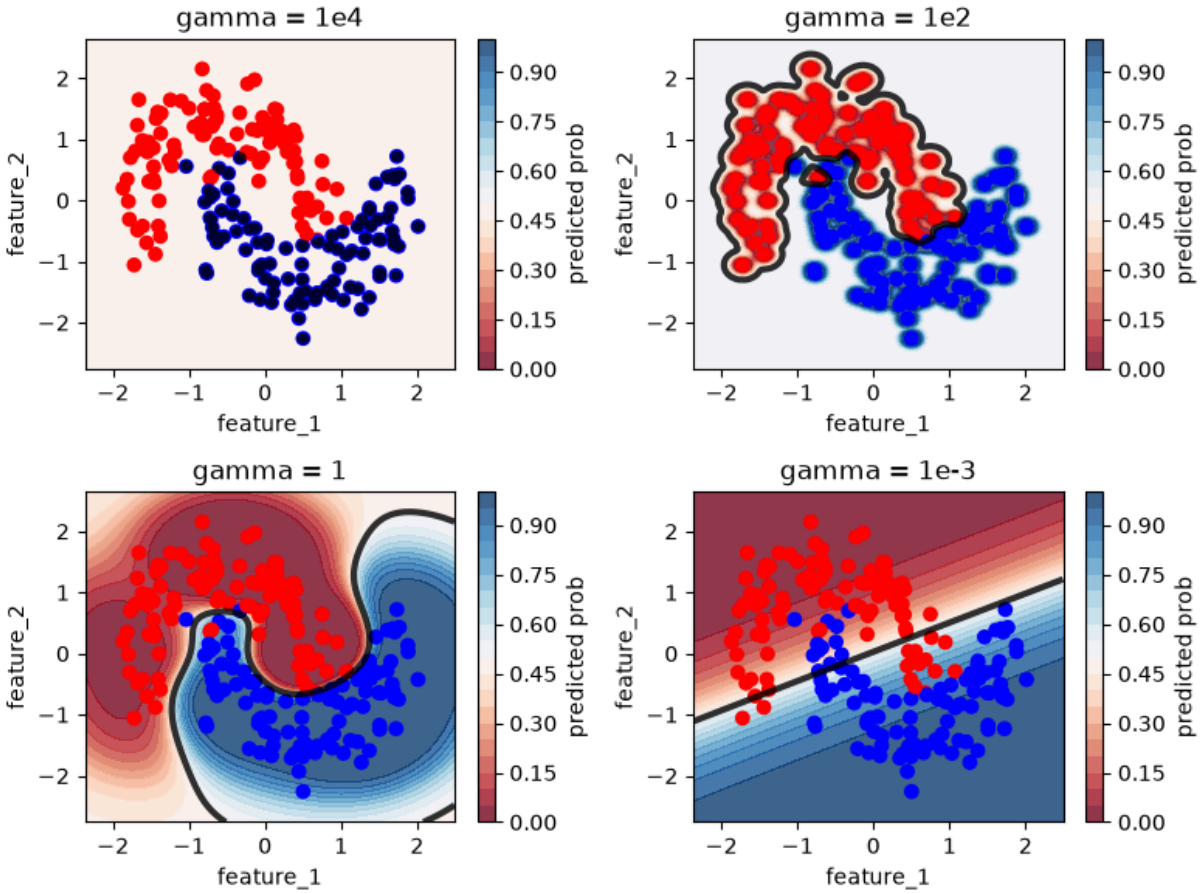
```
plt.ylabel('feature_2')
plt.title('gamma = 1e2')

plt.subplot(2,2,3)
clf = CalibratedClassifierCV(SVC(gamma = 1, C = 100), ensemble=False)
clf.fit(X,y)
Z = clf.predict_proba(np.c_[xx.ravel(), yy.ravel()])[:, 1]
# Put the result into a color plot
Z = Z.reshape(xx.shape)
plt.contourf(xx, yy, Z, cmap=cm, alpha=.8,vmin=0,vmax=1,levels=np.arange(0,1
plt.colorbar(label='predicted prob')
plt.contour(xx, yy, Z, alpha=.8,vmin=0,vmax=1,levels=[0.5],colors=['k'],line
plt.scatter(X[:, 0], X[:, 1], c=y,cmap=cm_bright)
plt.xlabel('feature_1')
plt.ylabel('feature_2')
plt.title('gamma = 1')

plt.subplot(2,2,4)
clf = CalibratedClassifierCV(SVC(gamma = 1e-3, C = 100), ensemble=False)
clf.fit(X,y)
Z = clf.predict_proba(np.c_[xx.ravel(), yy.ravel()])[:, 1]
# Put the result into a color plot
Z = Z.reshape(xx.shape)
plt.contourf(xx, yy, Z, cmap=cm, alpha=.8,vmin=0,vmax=1,levels=np.arange(0,1
plt.colorbar(label='predicted prob')
plt.contour(xx, yy, Z, alpha=.8,vmin=0,vmax=1,levels=[0.5],colors=['k'],line
plt.scatter(X[:, 0], X[:, 1], c=y,cmap=cm_bright)
plt.xlabel('feature_1')
plt.ylabel('feature_2')
plt.title('gamma = 1e-3')


plt.tight_layout()

plt.savefig('../figures/SVM_clf.png',dpi=300)
plt.show()
```



Quiz 2

Bias variance trade off

Which gamma value gives the best trade off between high bias and high variance? Work through the steps to answer the question.

- Use random_state = 42 where-ever necessary.
- Split X, y into X_train, X_val, y_train, y_val such that 70% of the points are in train.
- Fit SVC models with C = 1, and gamma = 1e-3, 1e-2, 1e-1, 1e0, 1e1, 1e2, 1e3 on the training set.
- Measure the validation accuracy for each gamma.
- Which gamma value gives the highest validation accuracy?

In [10]:
```
# add your code here
```

Non-linear ML algorithms - Support vector machines

By the end of this lecture, you will be able to

- Describe how linear SVMs work and what support vectors are
- Apply non-linear SVMs with radial basis functions (gaussian kernel)
- **Discuss what secondary considerations might go into selecting/excluding ML models**

Which ML algorithm to try on your dataset?

- other than predictive power, what else is important for you?
  - is it suitable for large datasets?
  - how the model behaves with respect to outliers?
  - does the prediction varies smoothly with the feature values?
  - can the model capture non-linear dependencies?
  - is the model easy to interpret for a human?
- here is a table to summarize things

ML algo	suitable for large datasets?	behaviour wrt outliers	non-linear?	params to tune	smooth predictions	easy to interpret?
linear regression	yes	linear extrapolation	no	l1 and/or l2 reg	yes	yes
logistic regression	yes	scales with distance from the decision boundary	no	l1 and/or l2 reg	yes	yes
random forest regression	so so	constant	yes	max_features, max_depth	no	so so
random forest classification	so so	step-like, difficult to tell	yes	max_features, max_depth	no	so so
SVM rbf regression	no	non-linear extrapolation	yes	C, gamma	yes	so so
SVM rbf classification	no	50-50	yes	C, gamma	yes	so so

- stakeholders will sometimes have special requests to include/exclude certain models
- i.e., I was told once to not even try tree-based algorithms on the dataset because they want a model that output smooth (meaning differentiable) predictions, not steps
- in certain domains (like banking) you might be limited to using interpretable models only (e.g., linear models or not too deep decision trees)

Mudcard

In [ ]: